

PlanComplexity

One-page repo-based summary of the current application.

What It Is

PlanComplexity is a Python desktop and command-line app for parsing RT Plan DICOM files and calculating plan complexity metrics. Repo evidence shows support for VMAT/IMRT, TOMO, and CyberKnife MLC workflows, plus CSV export and batch analysis.

Who It's For

Primary user: medical physicists or research users analyzing RT Plan complexity. The repo explicitly labels the GUI as research-use-only.

What It Does

- Desktop GUI for single-plan and folder-batch analysis.
- CLI entry points for single-file analysis and batch CSV export.
- Auto-detects analysis mode: VMAT/IMRT, TOMO, or CyberKnife MLC.
- Parses RT Plan DICOM data and computes modality-specific complexity metrics.
- Shows metadata, paper-oriented metric labels, notes, warnings, and support status.
- Exports result tables plus a companion CSV that documents output columns.

How It Works

- **Entry points:** `ucomx.py` launches Tkinter GUI; `main.py` runs single-file CLI; batch scripts call shared services.
- **UI layer:** `ucomx_gui.py` gathers file/folder input, mode, recursion, and renders tables plus warnings.
- **Service layer:** `ucomx_service.py` detects mode, parses plans, computes metrics, flattens outputs, summarizes results, and exports CSVs.
- **Parsing and metric engines:** Uses `pydicom`, `DicomParse/`, `tomo_parser.py`, `analysis_helpers.py`, and `tomo_metrics.py`.
- **Models and metadata:** `ucomx_models.py` defines modes/results; `metric_registry.py` supplies labels and descriptions.
- **Data flow:** RT Plan file/folder -> mode detection -> parser/metric engine -> `PlanAnalysisResult` -> GUI tables, console logs, or CSV exports.

How to Run

- Install dependencies: `python -m pip install -r requirements.txt`
- Launch the GUI: `python ucomx.py`
- Optional CLI: `python main.py --input-file path/to/plan.dcm`

If deployment, packaging, persistence, authentication, or network services were expected, they were not found in repo.